

AI ENGINEERING COHORT · CAPSTONE SUBMISSION

AppMind

Application Knowledge & Decision Intelligence Agent

Prabu Balasubramanian and Group 19 Team

September 2026

github.com/PrabuRepo/app-mind

Answering these questions today means finding the right person

- Application knowledge is scattered across docs, code, runbooks, and incident history
- Teams repeatedly ask: **why did this break, how does this work, what breaks if we change X**
- AppMind investigates on the team's behalf, producing an evidence-backed, auditable decision brief
- Escalates to a human when evidence is insufficient or contradictory — captures SME judgment, doesn't replace it

Deliberately narrow scope

One target application: **OrderFlow**

- Business / Functional
- Incident / RCA
- Impact Analysis

Real pipeline, synthetic data, verified guardrails



Sources

- **docs/** — 3 files (architecture, requirements, runbook)
- **incidents/** — 4 files, each a different planted trap
- **Code** — AST server (structure) + GitHub MCP (real text)
- RAG for docs/incidents — deliberately **not** RAG for code



Handling PII

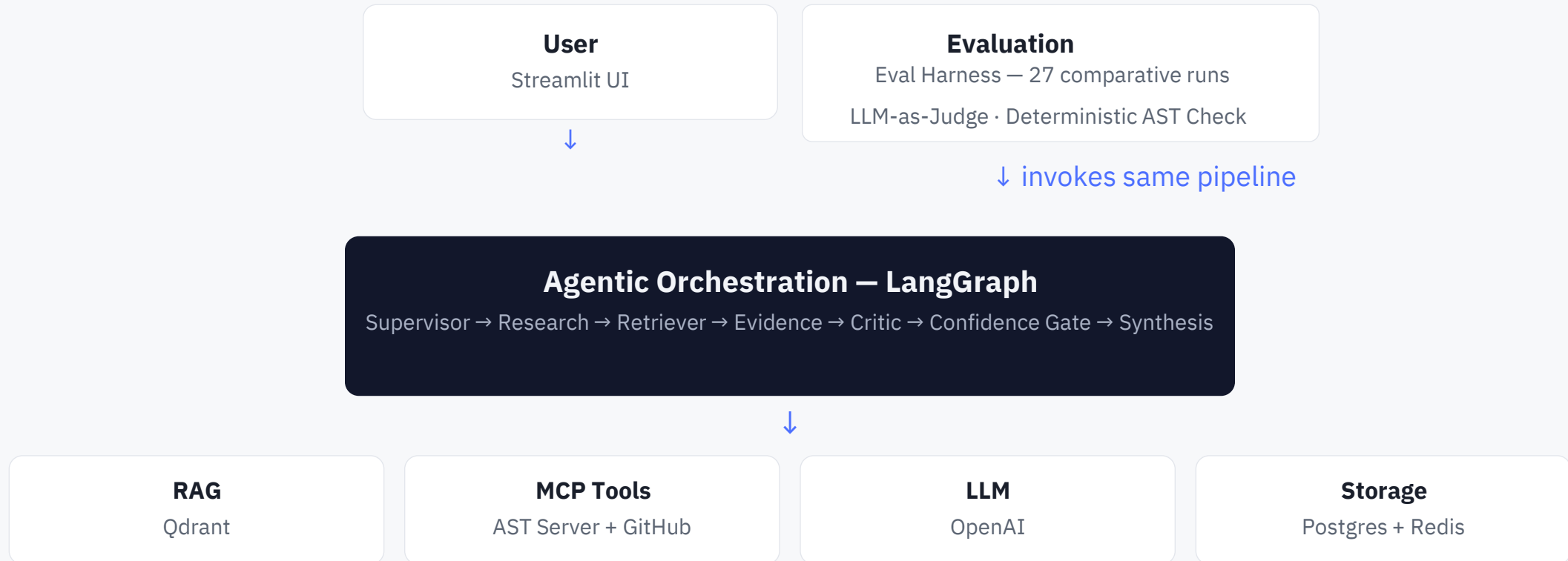
- All data is **entirely synthetic** — OrderFlow, incidents, and code are purpose-built
- Zero real user, customer, or production data anywhere in the corpus
- Nothing to anonymize, because nothing real was ever collected



Guardrails

- **Input** — blocks email/phone/SSN-shaped questions before any retrieval or LLM cost
- **Output** — blocks an uncited answer from shipping as a confident response
- Real, verified checks — not placeholder stubs

One orchestration layer, every source behind it



Chosen for the job, not for the résumé

Framework	Role in AppMind
LangGraph	Stateful orchestration — conditional routing per pipeline_mode, the Critic retry loop
Pydantic	Validated schemas for every inter-node payload — no silent malformed data
MCP SDK	Standard protocol for both code-access tools — custom AST server + GitHub
Qdrant	Vector search — separate docs / incidents collections
PostgreSQL	Investigation audit trail — one row per completed run
Redis	Investigation-lookup cache — repeat questions skip re-investigation
Streamlit	Single-page UI — calls the pipeline in-process, no separate API layer
OpenAI	LLM reasoning and embeddings — gpt-5.4-mini
Eval Harness	Custom-built (evals/ + llm_as_judge/) — direct SDK calls, not a framework — 27 comparative runs scored with LLM-as-Judge
Docker	Local infrastructure — Postgres, Qdrant, Redis

A mandatory comparison, scored two different ways

- **Custom-built harness** (evals/run_eval.py) — direct SDK calls, not a framework like promptfoo or ragas
- **3 pipeline_modes compared head-to-head** — baseline / critic_off / critic_on
- **9 questions** (3 demo + 6 backing) × 3 modes × 1 trial = **27 graph runs**
- Covers all 4 planted corpus traps — real bug, false alarm, doc contradiction, filler



LLM-as-Judge

Scores every subjective question for false-confidence, error-catch, and citation accuracy



Deterministic AST Check

Impact Analysis answers checked directly against the AST server's real dependency graph — zero LLM cost, zero variance



Error Handling

4 dedicated fault-injection cases — dead MCP, hung MCP, Qdrant down, empty retrieval

27 real graph runs, not a rehearsed demo

Metric	Baseline	Critic Off	Critic On
False-confidence rate	0.429	0.143	0.286
Error-catch rate	0.50	0.75	0.75
Citation accuracy	0.774	0.816	0.828
Mean tokens	1,294	2,542	9,367
Mean latency	3.8s	5.1s	14.7s

- **Error handling:** 4/4 fault-injection cases pass — every failure degrades to an honest escalation, never a crash
- **Test suites:** 14/14 AST server, 11/11 GitHub client, all unit suites passing
- **Honest finding:** false-confidence rate varied between two full eval runs (0.143 → 0.286) — single-trial variance, stated plainly, not smoothed over

A qualified win for the Critic — stated honestly, not oversold

0.75 vs 0.50

Error-catch rate — stable across both runs

~7x tokens

Real cost of the Critic vs. baseline

0.143 → 0.286

False-confidence rate — not yet stable at 1 trial

- The Critic can also be **over-cautious** — a genuine two-sided trade-off, not just a win
- Escalating instead of guessing, and reporting its own variance instead of hiding it, is itself validation of the core thesis

Deliberately scoped out, not forgotten

Near-term

- Run **--trials N** for statistically stable eval numbers
- Explore local Docker GitHub MCP (server-level read-only tool restriction)
- Make the AST server itself MCP-native — remove the local-checkout dependency
- UI V2 — pipeline_mode selector, critique-flag display, cost/latency footer

Explicitly deferred

- Reranker, HyDE, query-rewrite-as-a-step
- ANN algorithm comparison (HNSW vs. exact/IVF)
- Semantic memory reuse, richer guardrails, a second vector DB
- LangSmith evals — personal exploration vs. the custom harness